

Trabalho Final — Comparação de Classificadores

Grupo:

Leonardo Rodrigues Oliveira Saraiva — 12321BSI284
Murilo de Melo Barbosa Santos — 12321BSI245
Vitor Costa Andrade — 12321BSI255

1. Introdução e objetivos

Este trabalho tem como objetivo **aplicar, avaliar e interpretar** o desempenho de algoritmos de classificação supervisionada em **duas bases de dados reais e distintas**. Foram comparados três classificadores — **K-Nearest Neighbors (KNN)**, **Árvore de Decisão** e **Perceptron de Múltiplas Camadas (MLP)** — em duas bases com características propositalmente opostas: uma **grande e fortemente desbalanceada** (detecção de fraude em cartão de crédito) e outra **pequena e balanceada** (predição de doença cardíaca).

Para cada base foram realizadas as etapas de exploração dos dados, pré-processamento adequado a cada algoritmo, treinamento com diferentes parametrizações e avaliação por múltiplas métricas. Busca-se identificar **qual algoritmo se sai melhor em cada cenário**, se algum é **consistentemente melhor nas duas bases**, e **interpretar** os motivos por trás dos resultados.

2. Descrição das bases de dados

Ambas as bases foram obtidas no Kaggle e convertidas para o formato **Parquet** (colunar, compactado) para carregamento eficiente.

2.1. Base 1 — IBM Credit Card Fraud Detection

Campo	Valor
Nome	Credit Card Transactions (IBM — dados sintéticos, TabFormer)
Fonte / link	https://www.kaggle.com/datasets/ealtman2019/credit-card-transactions
Licença	Other (Kaggle)
Nº de instâncias	24.386.900
Nº de atributos	15 (14 preditores + 1 classe)
Atributo classe	Is Fraud? (booleano)
Balanceamento	Fortemente desbalanceado — 99,878% não-fraude vs 0,122% fraude (29.757 casos)

Atributos, tipos e valores ausentes:

Atributo	Tipo	Descrição	Ausentes
User	inteiro	ID do usuário	0%
Card	inteiro	ID do cartão	0%
Year / Month / Day	inteiro	Data da transação	0%
Time	hora	Horário da transação	0%
Amount	texto (\$134.09)	Valor (com símbolo \$)	0%
Use Chip	categórico	Swipe / Chip / Online	0%
Merchant Name	inteiro (hash)	Identificador do comerciante	0%
Merchant City	texto	Cidade do comerciante	0%
Merchant State	texto	Estado do comerciante	11,16%
Zip	numérico	CEP do comerciante	11,80%
MCC	inteiro	Categoria do comércio	0%
Errors?	texto	Erro na transação	98,41%
Is Fraud?	booleano	Classe — é fraude?	0%

2.2. Base 2 — Heart Failure Prediction

Campo	Valor
Nome	Heart Failure Prediction
Fonte / link	https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction
Licença	ODbL-1.0
Nº de instâncias	918
Nº de atributos	12 (11 preditores + 1 classe)
Atributo classe	HeartDisease (0 = não, 1 = sim)
Balanceamento	Balanceado — 55,3% (classe 1) vs 44,7% (classe 0)

Atributos, tipos e valores ausentes:

Atributo	Tipo	Descrição	Ausentes
Age	inteiro	Idade	0%
Sex	categórico	Sexo (M/F)	0%
ChestPainType	categórico	Dor no peito (ASY/ATA/NAP/TA)	0%
RestingBP	inteiro	Pressão em repouso (mmHg)	0%*
Cholesterol	inteiro	Colesterol (mg/dl)	0%*
FastingBS	inteiro (0/1)	Glicemia em jejum > 120	0%
RestingECG	categórico	ECG em repouso (Normal/ST/LVH)	0%
MaxHR	inteiro	Freq. cardíaca máxima	0%
ExerciseAngina	categórico	Angina por exercício (Y/N)	0%
Oldpeak	decimal	Depressão do segmento ST	0%
ST_Slope	categórico	Inclinação do ST (Up/Flat/Down)	0%
HeartDisease	inteiro (0/1)	Classe — doença cardíaca	0%

* Sem ausentes explícitos, mas há "**0 disfarçado de ausente**": `Cholesterol == 0` em **172 linhas** e `RestingBP == 0` em **1 linha** (valores fisiologicamente impossíveis), tratados como ausentes.

2.3. Justificativa da escolha (contraste entre as bases)

Aspecto	Base 1 (Fraude)	Base 2 (Cardíaca)
Tamanho	Muito grande (24M)	Pequena (918)
Balanceamento	Extremamente desbalanceado	Balanceado
Tipos de atributo	Numérico + categórico + texto	Numérico + categórico
Desafio principal	Escala + desbalanceamento	Encoding + "0 ausente"

3. Metodologia

3.1. Ferramenta / linguagem

Foi utilizada a linguagem **Python 3** com os pacotes **scikit-learn 1.6** (modelos, pré-processamento e métricas), **pandas** e **DuckDB** (manipulação e leitura de Parquet) e **PyYAML** (configuração). Todo o código é autoral e está no repositório citado no cabeçalho, organizado em: `explorar_dados.py` (exploração), `preprocessar.py` (pré-processamento) e `treinar.py` (treino/avaliação).

3.2. Exploração dos dados

Foi desenvolvido um *profiler* automático que calcula, por coluna, métricas de qualidade por tipo (numéricas: min, máx, média, desvio, percentis, IQR, outliers, assimetria, curtose; textuais: comprimento, % com aparência numérica), além de cardinalidade e % de ausentes. Os principais achados guiaram o pré-processamento: a base de **fraude** é fortemente desbalanceada (0,122%), tem

Errors? 98% ausente e Amount como texto; a base **cardíaca** é balanceada e apresenta os zeros fisiologicamente impossíveis em Cholesterol e RestingBP.

3.3. Pré-processamento aplicado

O pré-processamento foi **ajustado apenas no conjunto de treino** e aplicado ao teste, evitando *data leakage*. **Cada algoritmo recebeu um pré-processamento próprio**, justificado pela sua natureza:

Algoritmo	Ausentes	Normalização	Encoding	Justificativa
KNN	mediana / moda	Z-score	One-hot	baseado em distância → exige mesma escala; one-hot evita ordem artificial
Árvore	mediana / moda	Nenhuma	Ordinal	invariante à escala; ordinal mantém a dimensionalidade baixa
MLP	mediana / moda	Z-score	One-hot	treino por gradiente é sensível à escala → padronização essencial

Tratamentos específicos por base:

- **Cardíaca:** Cholesterol == 0 e RestingBP == 0 convertidos para ausente e imputados pela **mediana** do treino.
- **Fraude:** Amount convertido de texto para número; descartadas colunas de ID de alta cardinalidade (User, Card, Merchant Name, Merchant City, Zip, Time, Day) e a coluna Errors? (98% ausente); aplicado **undersampling** da classe majoritária na proporção **5 não-fraudes : 1 fraude** (a base original de 24M linhas é inviável de processar e degeneraria a acurácia), resultando em 178.542 instâncias.

3.4. Algoritmos de classificação

a) KNN — K-Nearest Neighbors (*algoritmo clássico, usado como baseline*)

1. Armazena todos os exemplos de treino (aprendizado "preguiçoso", sem modelo explícito);
2. para classificar um novo ponto, calcula a distância (euclidiana) a todos os exemplos;
3. seleciona os **K** vizinhos mais próximos;
4. atribui a classe majoritária entre eles (opcionalmente ponderada pela distância).

b) Árvore de Decisão

1. Escolhe o atributo/limiar que melhor separa as classes (menor impureza — **Gini** ou **entropia**);
2. divide os dados nesse ponto, criando ramos;
3. repete recursivamente até um critério de parada (profundidade, amostras mínimas por folha);
4. cada folha recebe a classe majoritária; a predição percorre a árvore da raiz à folha.

c) MLP — Perceptron de Múltiplas Camadas (rede neural) — algoritmo não visto em aula

Rede neural *feedforward* treinada por retropropagação. Passos:

1. **Arquitetura:** camada de entrada (1 neurônio por atributo), uma ou mais **camadas ocultas** e camada de saída;

2. **Inicialização** aleatória dos pesos e vieses;
3. **Propagação direta (forward)**: cada neurônio calcula $z = \sum(w_i \cdot x_i) + b$ e aplica uma **função de ativação** (ReLU nas camadas ocultas, logística na saída);
4. **Cálculo do erro** entre a saída prevista e a real por uma **função de perda** (entropia cruzada);
5. **Retropropagação (backpropagation)**: calcula o gradiente da perda em relação a cada peso, propagando o erro da saída para as camadas anteriores pela **regra da cadeia**;
6. **Atualização dos pesos** por um otimizador (**Adam**), na direção que reduz a perda;
7. **Repete** por várias épocas até convergir (`max_iter` ou tolerância atingida).

3.5. Divisão treino/teste

Holdout estratificado 70% treino / 30% teste (com `random_state = 42`), preservando a proporção das classes em ambos os conjuntos.

3.6. Medidas de avaliação e fórmulas

Sendo VP/VN = verdadeiros positivos/negativos e FP/FN = falsos positivos/negativos:

- **Acurácia** = $(VP + VN) / (VP + VN + FP + FN)$
- **Precisão** = $VP / (VP + FP)$
- **Recall (Revocação)** = $VP / (VP + FN)$
- **F1-score** = $2 \cdot (Precisão \cdot Recall) / (Precisão + Recall)$
- **ROC-AUC** = área sob a curva ROC (Taxa de Verdadeiros Positivos × Taxa de Falsos Positivos)

Como a base de fraude é desbalanceada, a **acurácia sozinha engana**; por isso o **F1-score** e a **ROC-AUC** foram adotados como critérios principais de comparação.

3.7. Parametrizações testadas (duas por algoritmo)

Algoritmo	Parametrização 1	Parametrização 2
KNN	K=3, voto uniforme	K=11, ponderado por distância
Árvore	Gini, profundidade máx. 5	Entropia, profundidade ilimitada (mín. 5 amostras/folha)
MLP	1 camada oculta (50), ReLU	2 camadas ocultas (100, 50), ReLU, $\alpha=0,001$

4. Resultados

Métricas no conjunto de **teste** (holdout 70/30 estratificado). Em **negrito**, o melhor valor de cada base.

4.1. Base Cardíaca (918 instâncias, balanceada)

Algoritmo	Parametrização	Acurácia	Precisão	Recall	F1	ROC-AUC
KNN	k=11, distância	0,880	0,900	0,882	0,891	0,941
KNN	k=3, uniforme	0,873	0,888	0,882	0,885	0,912
MLP	1 camada (50)	0,851	0,868	0,863	0,866	0,917
Árvore	entropy, ilimitada	0,812	0,848	0,804	0,826	0,882
Árvore	gini, prof.=5	0,837	0,842	0,869	0,855	0,860
MLP	2 camadas (100,50)	0,830	0,879	0,804	0,840	0,884

4.2. Base Fraude (178.542 instâncias após undersampling 5:1)

Algoritmo	Parametrização	Acurácia	Precisão	Recall	F1	ROC-AUC
MLP	1 camada (50)	0,968	0,916	0,888	0,902	0,990
MLP	2 camadas (100,50)	0,964	0,890	0,892	0,891	0,988
KNN	k=11, distância	0,958	0,912	0,829	0,868	0,972
KNN	k=3, uniforme	0,955	0,889	0,836	0,862	0,951
Árvore	entropy, ilimitada	0,956	0,885	0,842	0,863	0,950
Árvore	gini, prof.=5	0,936	0,854	0,743	0,795	0,927

5. Discussão dos resultados

Melhor algoritmo em cada base:

- **Cardíaca** → **KNN (k=11, ponderado por distância)**: F1 = 0,891 e ROC-AUC = 0,941.
- **Fraude** → **MLP (1 camada oculta)**: F1 = 0,902 e ROC-AUC = 0,990.

Tabela-resumo (melhor F1 por algoritmo em cada base):

Base	KNN	Árvore	MLP	Vencedor
Cardíaca	0,891	0,855	0,866	KNN
Fraude	0,868	0,863	0,902	MLP

Interpretação:

- **O tamanho da base foi decisivo.** Na base **pequena** (918 exemplos), o **KNN** — simples e sem parâmetros a estimar — superou o MLP, que tende a subaproveitamento com poucos dados. Na base **grande** (≈178 mil), o **MLP** teve dados suficientes para aprender fronteiras de decisão complexas e liderou.
- **Consistência:** o **KNN (k=11)** foi o mais consistente entre as bases — melhor na cardíaca e 3º na fraude (ROC-AUC 0,972, muito perto do topo). O MLP só se destacou na base grande.
- **A Árvore de Decisão** foi a mais fraca nas duas bases, embora competitiva na fraude com entropia e profundidade ilimitada.

- **Efeito da parametrização:** no KNN, K maior (11) com voto ponderado superou K=3 nas duas bases (mais robusto a ruído); no MLP, a rede mais simples (1 camada) superou a mais profunda.
- **A escolha da métrica importa:** na fraude, a Árvore (gini) tem acurácia 0,936 mas F1 de apenas 0,795 — confirmando que a acurácia isolada engana em base desbalanceada.
- **Custo computacional:** KNN e Árvore treinam em frações de segundo; o MLP na fraude levou ~2 minutos por configuração — um *trade-off* entre desempenho e custo.

6. Uso de LLMs

Declaração conforme o item 2.e do enunciado.

LLM utilizada: Claude (modelo Opus 4.8), por meio da ferramenta **Claude Code**.

Etapa	Uso da LLM	Prompt (resumo)
Obtenção das bases	Baixar do Kaggle e converter p/ Parquet	"Baixe esse dataset e coloque no meu diretório" / "quero apenas o dataset em .parquet"
Escolha da 2ª base	Sugerir base de classificação adequada	"procure outro dataset fácil no Kaggle para o trabalho universitário"
Exploração	Gerar o código de perfil das bases	"crie um código de escaneamento com métricas de qualidade por tipo e separação de categóricos por cardinalidade"
Pré-processamento	Gerar <code>preprocessar.py</code> + YAML configurável	"pré-processamento diferente por algoritmo; z-score, tratar ausentes; yaml configurável (normalização, encoding, ausentes)"
Treino/avaliação	Gerar <code>treinar.py</code> e executar KNN, Árvore e MLP	"cria/pegue da internet os algoritmos KNN, MLP, Árvore de decisão para treinar nos dois datasets"
Redação	Consolidar resultados e gerar este relatório	"consolida tudo" / "vamos gerar o pdf com todas as informações necessárias"

O grupo revisou e compreendeu o código e os resultados gerados, conforme exigido para a apresentação.

7. Referências

- Base 1 — Credit Card Transactions (IBM): <https://www.kaggle.com/datasets/ealtman2019/credit-card-transactions>
- Base 2 — Heart Failure Prediction: <https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction>
- scikit-learn — Pedregosa et al., *Scikit-learn: Machine Learning in Python*, JMLR 12, 2011. <https://scikit-learn.org>